Davante

8

ENTORNOS DE DESARROLLO

Diagramas de comportamiento

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Qué es un diagrama de comportamiento 4

/ 3. Tipos de diagrama de comportamiento 4

/ 4. Caso práctico 1: “Clases vs. comportamiento” 5

/ 5. Campo de aplicación 5

/ 6. Módulos para elaborar diagramas de comportamiento 6

/ 7. Caso práctico 2: “Elección de diagrama” 7

/ 8. Resumen y resolución del caso práctico de la unidad 8

/ 9. Bibliografía 8

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer los diagramas de comportamiento.

Aprender a aplicar los diferentes diagramas de comportamiento.

Saber la utilidad de los diagramas de comportamiento.

/ 1. Introducción y contextualización práctica

A la hora de modelar un sistema, necesitamos considerar la estructura de cada uno de los componentes que lo van a formar. Sin embargo, también es importante tener en cuenta cómo se van a comportar dichos componentes cuando la aplicación pase a un estado de ejecución.

Muchas veces, nos centramos únicamente en describir la estructura que tendrá cada uno de los componentes, sin considerar cómo se van a relacionar cuando se ejecuten. Por esta razón, es importante que utilicemos las capacidades que nos ofrece UML para establecer el flujo de comunicación de cada uno de los componentes de nuestro sistema.

Conocer esta forma de comunicación nos permitirá establecer una relación entre todos ellos en tiempo de ejecución para realizar un diseño completo de todo el sistema.

A medida que el sistema crezca, será más complicado realizar el modelado del flujo de ejecución. Por este motivo, en el lenguaje UML, se ofrecen diferentes tipos de diagramas que permiten representar el comportamiento que tendrán los elementos durante la ejecución. Como su propio nombre indica, este tipo de diagramas se denominan **diagramas de comportamiento**.

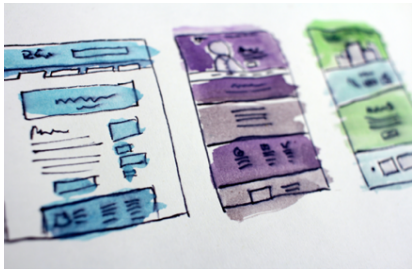


Fig. 1. Cada diagrama UML tiene una finalidad bien definida.

Escucha el siguiente audio, en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.

Audio Intro. "Diagramas de comportamiento y utilización"
https://bit.ly/3fTscQ2

/ 2. Qué es un diagrama de comportamiento

A la hora de desarrollar una nueva aplicación o bien al realizar alguna modificación en una existente, lo normal es que los desarrolladores consulten los requisitos solicitados por el cliente.

Es muy normal que los desarrolladores no tengan contacto directo con los clientes, sino con los comerciales de nuestra empresa, que serán los que tendrán que intervenir en la toma de requisitos junto con un asesoramiento técnico de analistas o jefes de proyecto. De esta forma, los clientes de los desarrolladores serían los propios compañeros de la empresa, encargados de tomar requisitos.

En este contexto, un diagrama de comportamiento permite simplificar cómo debe comportarse el sistema ante la nueva funcionalidad que solicitan los clientes. Podríamos decir que un diagrama de comportamiento expresa la secuencia de estados por la que va a pasar un objeto de nuestro sistema a lo largo de su vida para dar respuesta a los eventos que tengan lugar en el sistema.

Gracias a los diagramas de comportamiento, podemos representar de forma visual los procesos que tienen lugar en nuestro programa. Normalmente, se utilizan para visualizar y especificar, a la vez que documentar, cambios constantes en el sistema. La complejidad de los diagramas de comportamiento dependerá del contexto y del proceso que estén representando en cada momento.

En los diagramas de comportamiento, se representa la relación que tienen los objetos de nuestro sistema, pues estamos utilizando programación orientada objetos. Dada la naturaleza de este tipo de programación, es importante saber cómo se comportan los objetos cuando se utiliza cada una de sus funcionalidades y se relacionan con otros objetos del entorno.

UML ofrece mecanismos para crear los diferentes diagramas de comportamiento que hagan falta en nuestro modelado para proporcionar una documentación y la información necesaria a los desarrolladores y analistas.



Fig. 2. Existen diferentes tipos de diagramas de comportamiento.

Audio 1. "Qué es un diagrama de comportamiento y tipos"
https://bit.ly/30PwlQK

/ 3. Tipos de diagrama de comportamiento

UML contempla diferentes tipos de diagrama de comportamiento que pueden ser utilizados en las diferentes fases de creación de la documentación y del modelado, para aportar la información necesaria y aclarar aquellos aspectos de una manera correcta.

Cada diagrama de comportamiento permite observar el sistema desde un punto de vista diferente. Por esta razón, normalmente, utilizamos más de un tipo de programa de comportamiento para modelar el sistema y evitar posibles ambigüedades.

Dependiendo de la bibliografía consultada, el número de diagrama de comportamientos varía, pues algunos autores consideran unos diagramas y otros autores consideran otros tipos de diagramas. De manera general, podríamos decir que existen los siguientes diagramas de comportamiento:

- **Diagrama de casos de uso.** Este tipo de diagrama permite mostrar las diferentes operaciones que pueda realizar el sistema desde el punto de vista del usuario. Además, es capaz de indicar la relación que tiene el sistema con el entorno y otras posibles operaciones necesarias para realizar la tarea actual.

- **Diagramas de secuencia.** Se trata de otro tipo de diagrama de comportamiento, que permite describir las diferentes interacciones que tienen los objetos de nuestro sistema a lo largo del tiempo. Normalmente, suelen acompañar a los diagramas de casos de uso para ampliar el grado de detalle.

- **Diagramas de estados.** Permiten indicar los diferentes estados por los que va pasando un objeto dentro del flujo de ejecución del software. También permiten observar las condiciones necesarias para que un objeto pueda cambiar de estado o volver al estado anterior.

- **Diagramas de actividad.** Es una especialización de los diagramas de estado que permite diferenciar diferentes tipos de estados. En este diagrama interesan los estados de acción y las transiciones que ocurren al finalizar dicha tarea.

Vídeo 1. "Visual Paradigm. Generación de código a partir de diagrama"
https:/bit.ly/2Ooa3OM

/ 4. Caso práctico 1: "Clases vs. comportamiento"

Planteamiento: En nuestra empresa, es necesario realizar el modelado completo de un sistema cuando se está ejecutando para poder informar a los desarrolladores qué tareas está llevando a cabo. Desde el departamento de calidad, dicen que, para esto, se debe crear un diagrama de clases.

Nudo: ¿Qué te parece esta decisión?

Desenlace: Los diagramas de clase son diagramas de una gran potencia, pero no permiten mostrar el flujo de ejecución de un sistema ni las interacciones que tiene una clase con otra durante la ejecución de la aplicación.

Para esto, es mejor introducir un diagrama de comportamiento que muestre cómo se comporta la aplicación en tiempo de ejecución. Los desarrolladores necesitarán este tipo de diagramas cuando surja un error. Desde el punto de vista de calidad, el diagrama de comportamiento sirve para plantear diferentes tipos de pruebas que puedan mejorar la calidad de nuestro sistema.



Fig. 3. Una buena documentación ayuda a tener mejores desarrollos.

Conociendo también los diagramas de clase, podemos saber qué funcionalidades tiene un sistema sin tener que acudir al código. Si lo usamos en conjunto con los diagramas de comportamiento, podremos tener unas pruebas más sólidas que comprueben mayor cantidad de funcionalidades.

Como podemos observar, el modelado del sistema no solo servirá a los desarrolladores, sino que también ayudará a otros departamentos a desempeñar una tarea más centrada en el producto que hemos desarrollado.

/ 5. Campo de aplicación

Los diagramas de comportamiento se utilizan cuando queremos mostrar el flujo de ejecución y la interacción que tiene nuestra aplicación en referencia a cada uno de los módulos que la forman. La idea es mostrar de manera dinámica cómo se comporta el sistema y reflejar cómo las diferentes actividades, que tienen lugar en el sistema, cambian en un periodo de tiempo determinado.

Gracias a los diagramas de comportamiento, podemos entender visualmente qué tareas deben realizar los diferentes procesos y cuáles deberían realizarse en otros. Con la utilización de estos diagramas, también podemos mostrar el funcionamiento general del sistema a través de la visualización de los diferentes procesos que ejecuta.

Existen diferentes tipos de comportamientos que permiten crear un diagrama de este estilo. Por ejemplo, se puede crear un diagrama cuando ocurre un evento, al tener lugar una señal, cuando se dispara un trigger, al recibir una petición en un búfer, etc.

En la especificación de UML 2.4.1 se definen dos tipos de comportamientos, principalmente: comportamiento de ejecución y comportamiento emergente.

- El comportamiento de **ejecución** es un tipo de comportamiento que se lleva acabo cuando el objeto está en su flujo de ejecución. Normalmente, este comportamiento está relacionado directamente por la implicación de alguno de los métodos que ofrece a través de su interfaz. Este tipo de comportamiento tiene acceso a las características estructurales de los objetos que, normalmente, está vinculado con un método.

- El comportamiento **emergente** es aquel que tiene lugar por la interacción de uno o más objetos que participan en la relación. Los objetos participantes pueden ser parte de un objeto compuesto por otros. En este caso, se tiene una relación indirecta, lo cual puede complicar el diagrama de comportamiento.

Una de las características del comportamiento es que no tiene una notación particular más allá de la relación y del flujo que se establece con las clases.

Enlaces de interés...

Visualiza el siguiente vídeo donde podrás encontrar un ejemplo de modelado de comportamiento:
https://www.youtube.com/watch?v=ruoylFi7Irs

/ 6. Módulos para elaborar diagramas de comportamiento

Como hemos visto anteriormente, los diagramas de comportamiento pueden ser de diferentes tipos. Esto incluye diagramas de casos de uso, diagrama de actividad, etc.

Al tratarse de un tipo de diagrama ampliamente utilizado, la mayor parte de las herramientas CASE soportan este tipo de diagramas. Podemos tomar como referencia herramientas que vimos en el tema 6 (integradas en IDE o no) como *Papyrus UML, StarUML o Rational Rose*.

A continuación, se muestran ejemplos de diagramas de comportamiento creados con herramientas CASE.

Como se puede observar en la figura 4, el diagrama de secuencia permite observar la interacción que existe entre la clase cliente y la compra online de libros. Gracias al diagrama de secuencia, que pertenece a un diagrama de comportamiento, se puede ver fácilmente el flujo de ejecución que tiene que realizar un cliente para realizar una compra web.

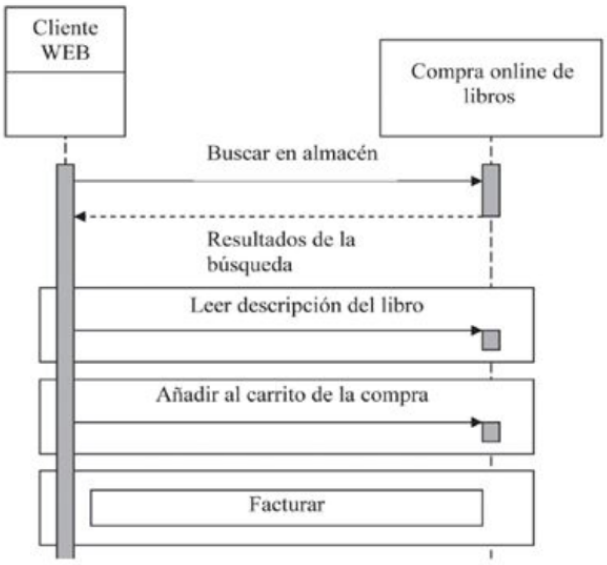


Fig. 4. Diagrama de secuencia.

En la siguiente figura se muestra un ejemplo del diagrama caso de uso, en el que se puede especificar la funcionalidad que tiene el sistema con un alto nivel de abstracción. En este caso, no es necesario indicar el flujo de ejecución, sino las tareas que se pueden realizar.

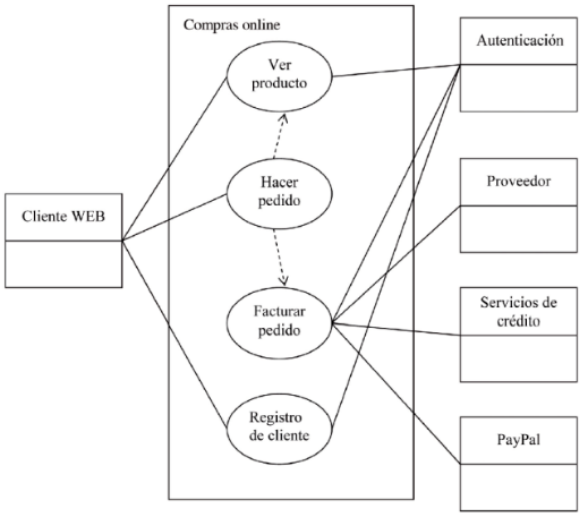


Fig. 5. Diagrama de caso de uso.

/ 7. Caso práctico 2: "Elección de diagrama"

Planteamiento: En nuestra empresa se plantea la creación de un diagrama de comportamiento para observar el flujo de ejecución que tiene lugar desde que el usuario hace clic en el botón RESERVAR hasta que realmente finaliza todo el proceso. Según nuestro jefe, lo que nos están pidiendo es que realicemos un diagrama de casos de uso.

Nudo: ¿Crees que este enfoque es correcto? ¿Existe alguna alternativa mejor?

Desenlace: Como hemos visto a largo del tema, los diagramas de comportamiento permiten mostrar el comportamiento del sistema ante la interacción de los usuarios o de unas clases con otras.

Según nos piden en el enunciado, el diagrama que necesitamos tendría que mostrar el flujo de ejecución desde que sucede un evento, en este caso la reserva, hasta que se produce el fin de la acción. Si utilizamos un diagrama de casos de uso, lo que estamos mostrando es la funcionalidad que tiene el sistema, y ya sabemos que podemos realizar una reserva de un producto. Por lo tanto, este enfoque no sería correcto, ya que tendríamos que considerar añadir un diagrama que permita mostrar el flujo de ejecución.



Fig. 6. Elegir un mal diagrama dificulta la visibilidad de las funcionalidades.

En el caso de tener una solución en la que hay varios procesos que están en ejecución, tendríamos que mostrar ambas perspectivas para ver todo el flujo de cada uno de los procesos que estén involucrados en la tarea que nos solicita. De esta forma, un ejemplo de diagrama de comportamiento que podríamos utilizar sería el diagrama de secuencia para mostrar cada una de las tareas que hay que desarrollar en cada uno de los estados por los que pasa esta acción.

/ 8. Resumen y resolución del caso práctico de la unidad

A largo de este tema hemos presentado las principales **características de los diagramas de comportamiento** para mostrar el flujo de ejecución que tienen los componentes que conforman una aplicación.

Gracias a los diagramas de comportamiento, los desarrolladores, así como nuestros clientes, podrán entender de una manera más sencilla cómo se comporta el sistema. Desde el punto de vista de la documentación y del futuro mantenimiento, incluir un diagrama de comportamiento de cada uno de los módulos del sistema puede ser una tarea que ahorre mucho tiempo a la hora de comprender qué está pasando ante un determinado fallo sistémico del sistema. Cuando vayamos a elegir un diagrama de comportamiento, tenemos que tener muy claro su uso, sabiendo qué es lo que queremos mostrar en dicho diagrama.

Resolución del caso práctico de la unidad

Como hemos visto en el tema, un diagrama de comportamiento permite mostrar cómo **se comporta el sistema bajo determinadas circunstancias**. Por ejemplo, un diagrama de comportamiento nos permite mostrar cómo se comporta el sistema en un estado dinámico.

La propuesta que nos hace nuestro responsable a la hora de crear un diagrama de secuencia es correcta. Sin embargo, dadas las características que pueden tener los diferentes diagramas, quizás es mejor tener un enfoque con múltiples diagramas para poder explicar y documentar mejor el resto del sistema.

Con un enfoque multidisciplinar, podríamos tener una documentación más enriquecida y reducir la posibilidad de cometer errores. Un problema de tener múltiples diagramas es que hay que realizar el mantenimiento por duplicado o triplicado, dependiendo del número de diagramas que tengamos. Por ello, es necesario llegar a un consenso entre el número de diagramas y las funcionalidades a representar.



Fig. 7. Tener un número de diagramas elevado no será ágil.

/ 9. Bibliografía

Juan, J. y Resusta, L. (2014). *UML práctico: aprende UML paso a paso*. España.

Pender, T., McSheffrey, E. y Varveris, L. (2003). *UML Bible*. Indianapolis: Wiley Pub.